

CASE STUDY · DATA SCIENCE

Prevención de fraude orientada a ganancia

78,9%

de la ganancia ideal (**63,4% aprobando todo**)

+232 mil sobre 150.000 transacciones de 45 días · un modelo que estima el riesgo y una política que decide con el costo de cada error



Juanfe Martínez

DATA SCIENCE · MACHINE LEARNING · PREVENCIÓN DE FRAUDE

Dataset · 150.000 transacciones

Código · github.com/JuanFeDS/proyecto

Cada decisión tiene un costo: aprobar un fraude o rechazar una venta

Cada venta legítima aprobada deja el 25% de su monto; cada fraude aprobado se pierde completo. Por eso el modelo no se mide por aciertos sino por **ganancia**: estima la probabilidad de fraude de cada transacción, y una política la rechaza cuando el riesgo supera lo que cuesta perder la venta.

4x aprobar un fraude cuesta cuatro veces lo que deja una venta	20%, no 50% → umbral de la política: se rechaza si $P(\text{fraude}) \geq 20\%$	78,9% → de la ganancia ideal, frente a 63,4% de aprobar todo	+232 mil → de ganancia sobre 150.000 transacciones de 45 días
+232 mil ganancia adicional frente a aprobar todo (+24%)	59% del dinero en fraude evitado: 325 mil de 547 mil	94,5% de las transacciones se siguen aprobando	

A cambio, se rechaza el 3% de las ventas legítimas: 93 mil de margen resignado.

DEPENDENCIA CRÍTICA • EL SCORE

La variable **score** es la que más ganancia aporta: desordenarla cuesta 8,3 puntos, más que cualquier otra. **El resultado solo es válido si el score está disponible antes de la decisión y se calcula sin información posterior.** Sin él, el baseline pierde 1,3 puntos y sigue superando ampliamente a las reglas actuales.

Cuatro decisiones que hacen confiable el resultado

● Métrica de negocio

Todo se optimizó y se evaluó en ganancia, no en accuracy: aprobar todo acierta el 95% y aun así deja pasar 547 mil en fraude.

● Optimismo medido

El ajuste de hiperparámetros se volvió a medir con otra partición: aporta +0,4 puntos, no los +0,7 que prometía la búsqueda.

● Variables sin la propia etiqueta

Las variables que resumen otras transacciones se ajustan solo con los folds de entrenamiento, como en producción.

● Paridad laboratorio–producción

El mismo pipeline se evalúa, se registra en MLflow y decide en la API desplegada en la nube.

El error caro es aprobar un fraude

HALLAZGO

Con un costo 4 veces mayor para el fraude aprobado, acertar más no es lo mismo que ganar más.

El dataset tiene **150.000 transacciones** entre el 8 de marzo y el 21 de abril de 2020: monto, fecha, un **score** de riesgo calculado por otro sistema, 15 variables anonimizadas (a - p) y la etiqueta de fraude (5% de los casos).

DECISIÓN	TRANSACCIÓN LEGÍTIMA	TRANSACCIÓN FRAUDULENTA
Aprobar	+25% del monto	-100% del monto
Rechazar	0 · se pierde la venta	0 · se evita la pérdida

El éxito se mide como porcentaje de la **ganancia ideal**: la que obtendría alguien que supiera de antemano qué transacciones son fraude (1,50 millones sobre este dataset). Aprobar todo deja 948 mil, el 63,4%, pese a acertar el 95% de las veces. Montos en la unidad monetaria del dataset.

Hipótesis de partida

Seis hipótesis sobre cómo suele comportarse el fraude en pagos online guiaron el análisis. No todas pesan igual: tres definen cómo se decide y se modela, y tres se traducen en variables.

CAMBIAN LA DECISIÓN O EL MODELO

- H1 Acertar más no es ganar más**
El costo asimétrico debería llevar el umbral de rechazo muy por debajo del 50%.
- H5 El score ayuda, pero no alcanza solo**
Debería ser la señal más fuerte, pero combinado con el resto decidir mejor.
- H6 El fraude se repite por categoría**
Una variable con miles de categorías tendría señal, con riesgo de que el modelo la memorice.

SE TRADUCEN EN VARIABLES

- H2 El fraude busca montos altos**
Las transacciones caras deberían concentrar el riesgo y, sobre todo, las pérdidas.
- H3 Menos historial, más fraude**
Las variables que midan antigüedad o actividad deberían ser más bajas en los fraudes.
- H4 El fraude tiene horarios**
La madrugada, con menos actividad legítima, debería concentrar más fraude.

Resultado del contraste

#	RESULTADO
H1	PREMISA Se deduce de los costos (corte 20%); con 50%, 4,7 puntos menos
H5	CONFIRMADA Como regla única, el score deja el 67,4%; el modelo que lo combina con el resto llega al 78,9%
H6	CON Matices Hay señal real, pero en su forma original el modelo la memorizaba (sección 02)
H2	CONFIRMADA El 20% más caro tiene 7,3% de fraude y concentra el 82% del dinero perdido, frente al 67% del volumen
H3	CON Matices Tres variables triplican el fraude en su cuartil más bajo; que midan historial es una interpretación
H4	CONFIRMADA Entre 0 y 4 h el fraude va de 10% a 17%, sobre el 3,2% del volumen

H5 · El score es la señal más fuerte, pero no decide solo

Fraude según el score de riesgo

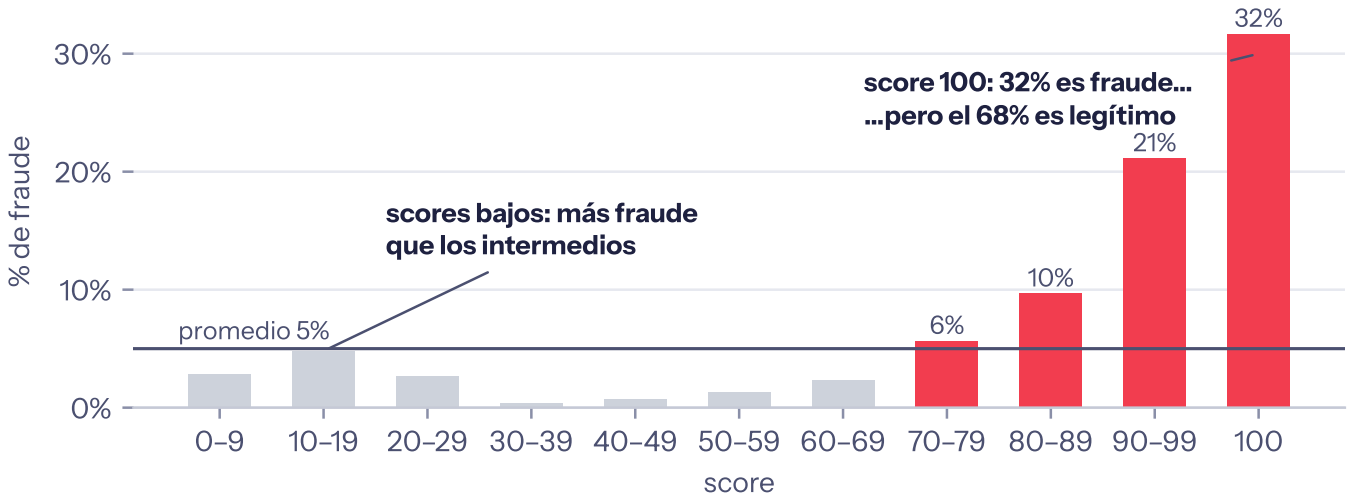


Figura 1 · Usado como regla, el score rechazaría demasiadas ventas legítimas.

La relación no es lineal: los scores bajos tienen más fraude que los intermedios, así que parece medir un riesgo general más que el fraude en sí. Descartamos una filtración de la etiqueta: la relación es gradual e imperfecta, y el resto de las variables explica solo el 16% de su valor.

H2 · El dinero perdido se concentra en los montos altos

Fraude y pérdidas por tramo de monto

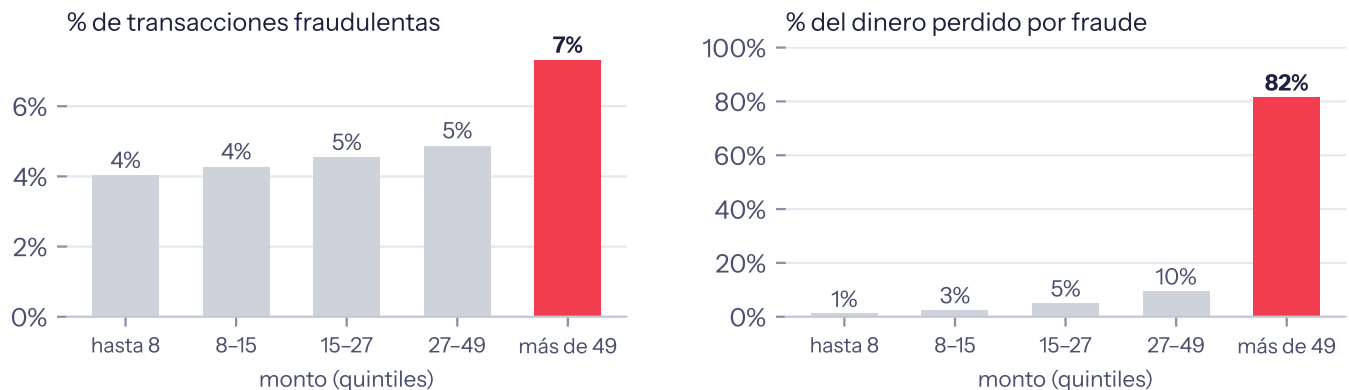


Figura 2 · El 20% más caro tiene 7,3% de fraude (4-5% en el resto) y concentra el 82% de la pérdida, frente al 67% del volumen.

Los fraudes tienen un monto promedio de 73, frente a 42 de las legítimas. Por eso los fraudes pesan más en la pérdida que en el conteo.

H4 · Más fraude en la madrugada, sobre poco volumen

Fraude por hora del día



Figura 3 · La tasa es alta, pero sobre pocas transacciones: el 17% de las 2 h sale de 638 casos. Como variable aporta poco: quitar **hora** del modelo final cuesta 0,26 puntos.

H3 · Menos historial, más fraude

Seis variables anonimizadas (**f** , **l** , **m** , **d** , **e** , **h**) son más bajas en los fraudes, pero con fuerza muy distinta. En **f** , **l** y **m** el cuartil más bajo tiene cerca de 9,5% de fraude frente a 3,4% en el resto (AUC univariado 0,66–0,68); en **d** la diferencia es menor (7,9% frente a 4,0%), y en **e** y **h** es débil (AUC 0,54–0,56). Varias se comportan como contadores con tope (**d** se acumula en 50), así que es razonable leerlas como antigüedad o actividad y el patrón apuntaría a **entidades con menor historial**. Por eso el veredicto es *con matices*: la relación con el fraude está medida, pero que sean historial es una interpretación, porque las variables están anonimizadas.

Lo que no esperábamos · tres variables que juntas definen perfiles

Fraude según el perfil o / n / p

Combinación de o, n y p ("nulo" = sin dato), perfiles con al menos 100 transacciones.

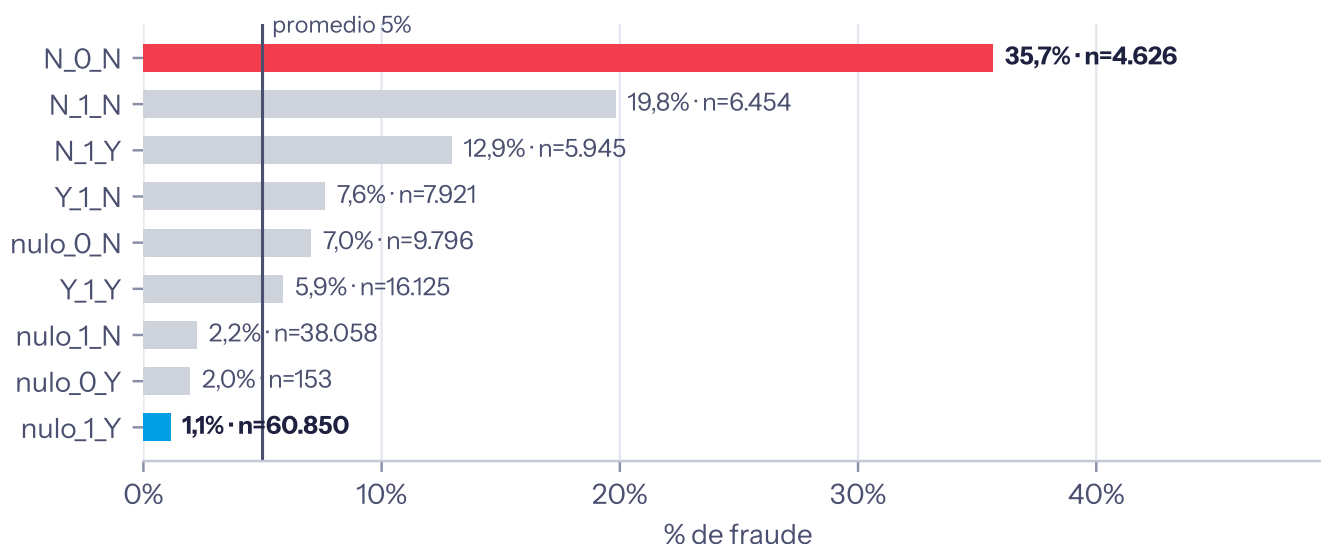


Figura 4 · De 1,1% a 35,7%: la variable con más señal individual del dataset.

Un dato vacío en **o** , que podría tratarse como un error, resultó la señal de **menor** riesgo: el perfil más frecuente (41% del volumen) tiene 1,1% de fraude. En el modelo, sin embargo, el perfil no aporta: quitarlo no cambia la ganancia, así que el modelo final no lo usa. LightGBM aprende la combinación por su cuenta, y **o** pasa a ser la segunda variable en valor.

De datos crudos a señales que no memorizan

HALLAZGO

La variable **j** tiene señal real, pero en su forma original el modelo la memorizaba.

150k

transacciones

5%

de fraude

45 días

marzo-abril 2020

8.324

categorías en **j**

Alertas del dataset

5% exacto de fraude	→ probable muestreo: con la tasa real hay que corregir las probabilidades
Hasta 72,6% de datos vacíos	→ el vacío es información (en 0 , la señal de menor riesgo): no se imputa
74% Brasil · 21% Argentina	→ hay países con 1 o 2 transacciones: los de menos de 100 se agrupan en "Otros"
Fraude diario entre 2,2% y 6,6%	→ el riesgo varía en el tiempo: hay que monitorearlo en producción
27% de las categorías de j aparece una vez	→ siempre llegarán categorías nuevas: necesitan un valor por defecto

Transformaciones

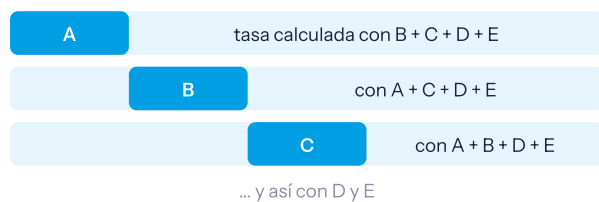
VARIABLE	CONSTRUCCIÓN	HIPÓTESIS
Tasa de fraude de j	Target encoding suavizado, calculado sin la etiqueta de la propia fila	H6 · conserva la señal sin memorizar
Frecuencia de j	Transacciones de la categoría en entrenamiento	H6 · 2,8% de fraude en categorías de 1 caso, 7,0% en las de más de mil
Perfil o/n/p	Combinación de las tres, con el vacío como categoría	Hallazgo inesperado · AUC univariado 0,799 · descartado: 0 ya lo captura
País agrupado	Países con menos de 100 transacciones → "Otros"	Evita tasas de países con 1 o 2 casos
Hora	Hora del día	H4 · quitarla cuesta 0,26 puntos

Se construyeron 18 variables. Cada una se evaluó por su señal individual (con una variable aleatoria como piso de ruido) y por su aporte al modelo, comparando conjuntos fold a fold. Las otras 13 (ratios de historial, montos redondos, actividad reciente, día de la semana) no mejoraron el modelo: con ellas se obtenía lo mismo con 33 variables en lugar de 20 (19 en el modelo final, sin el perfil o/n/p).

Cómo se calcula la tasa de fraude de j sin filtrar la etiqueta

ENTRENAMIENTO

cada fila se codifica con los otros 4 folds internos



PREDICCIÓN (validación o producción)

se usa la tabla ajustada con todo el entrenamiento



$$\text{tasa}(j) = (\text{fraudes de } j + 20 \cdot \text{tasa global}) / (\text{transacciones de } j + 20)$$

el suavizado acerca a la media las categorías con pocos casos

Impacto medido: con la etiqueta de la propia fila, la variable parece mucho más fuerte (AUC 0,783); sin ella, cae a 0,645. La diferencia era memorización, y en validación se recalcula dentro de cada fold externo.

Figura 5 · Target encoding out-of-fold en entrenamiento y tabla fija para predecir.

REGLA APLICADA A TODAS LAS VARIABLES CON ESTADO

En el pipeline final, la tasa y la frecuencia de j y la lista de países frecuentes se aprenden solo con los folds de entrenamiento y se aplican tal cual a los de prueba; validado así, reproduce el 78,9%. En la comparación de variables, la frecuencia y los países se calcularon con todo el dataset (sin etiquetas). Como los folds son aleatorios, es un cálculo out-of-fold, no temporal: una tasa puede usar etiquetas posteriores.

Lo que movió la ganancia no fue el algoritmo

HALLAZGO

Tres algoritmos empatan dentro del ruido; el valor vino de las variables y de la política de decisión.

Cada etapa cambió un solo elemento y se comparó contra la anterior en los mismos cinco folds, fold a fold. Referencias sin machine learning: aprobar todo (63,4%) y rechazar desde score 90 (67,4%). El baseline, LightGBM por defecto con las variables originales, llegó al 77,7%.

Variables • +1,2 puntos, casi todo por tratar bien j

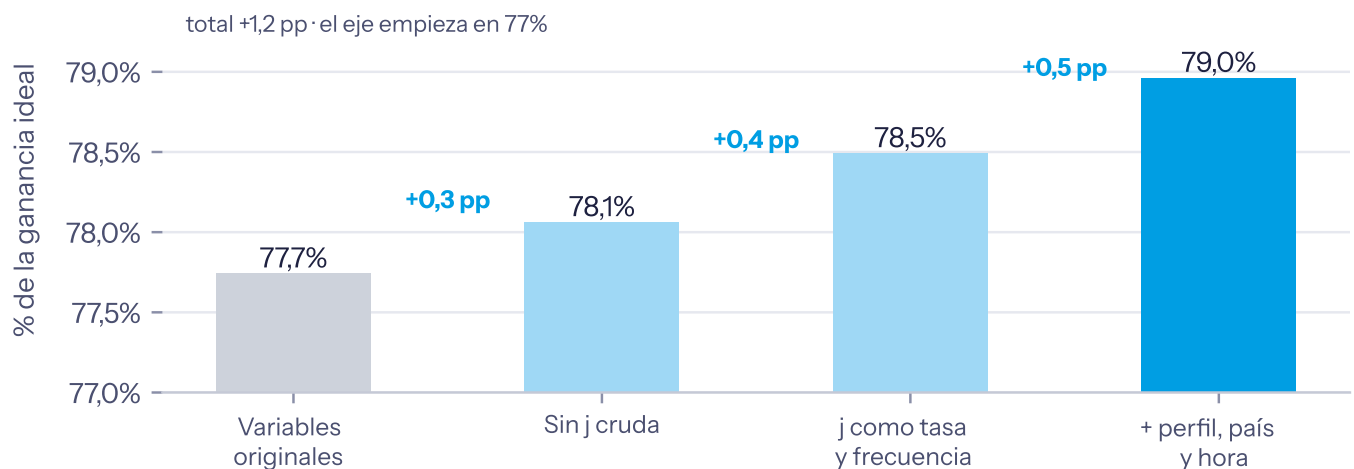


Figura 6 • Aporte de cada paso sobre el baseline, con LightGBM por defecto.

Algoritmos • un empate técnico

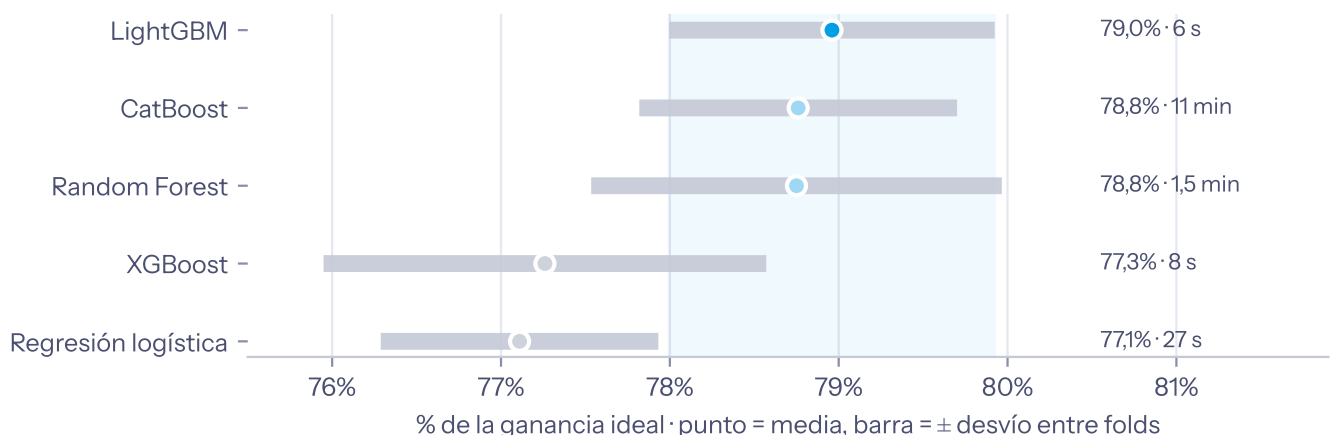


Figura 7 • Las tres mejores medias caben en la banda de ± 1 desvío de LightGBM.

Se eligió **LightGBM** por su desempeño equivalente al de CatBoost y Random Forest, una validación 100 veces más rápida que CatBoost y manejo nativo de nulos y categóricas.

Ajuste, pesos y simplificación

- **Optuna, 50 combinaciones** buscando la ganancia: con los folds de la búsqueda, +0,7 puntos; con otra partición (semilla 7), +0,4. La diferencia es optimismo por elegir y evaluar con los mismos datos. 39 de las 50

combinaciones superaron el 79%: una zona estable, no un óptimo frágil.

- **Pesos por monto:** no mejoran. Con peso proporcional al monto la ganancia baja 0,5 puntos, porque unas pocas transacciones muy caras dominan el aprendizaje. El costo se incorpora mejor en la política de decisión que en el entrenamiento.
- **Simplificación:** quitar el perfil o/n/p no cambia la ganancia (+0,1 puntos, mejor en solo 2 de 5 folds): la misma ganancia con una variable menos, porque LightGBM ya combina **o** , **n** y **p** . Quitar **hora** , en cambio, cuesta 0,26 puntos: se queda.

Sistema final: modelo + política

Modelo	LightGBM tuneado (481 árboles, aprendizaje lento, hojas de al menos 262 casos) que estima P(fraude transacción) con 19 variables
Política	Rechazar si $P(\text{fraude}) \geq 0,20$, el umbral teórico: sale de los costos, sin estimar nada, y los umbrales alternativos probados no se distinguen de él
Entrenamiento	150.000 transacciones, ~15 segundos

La política de decisión **vale más que el algoritmo**

HALLAZGO

Cortar según el costo aporta 4,7 puntos frente al 50% por defecto; elegir entre los mejores algoritmos, 0,2.

Todas las cifras son predicciones **out-of-fold**: cada transacción fue evaluada por un modelo que no la vio. El modelo final se validó con una segunda partición de cinco folds (semilla 7), distinta de la del ajuste de hiperparámetros. Con ella también se decidieron los pesos, las variables y el umbral, así que la prueba limpia es temporal: la última semana, con ajuste y umbral elegidos solo con el pasado.

Ganancia de cada alternativa sobre la ganancia ideal



Figura 8 • +15,5 puntos sobre aprobar todo y +11,5 sobre la regla por score.

El umbral sale del costo, no del 50%

El modelo no dice "fraude": entrega una probabilidad, y dónde cortar sale de la matriz de costos. Aprobar una transacción de monto m conviene si $0,25 \cdot m \cdot (1-p) > m \cdot p$, es decir, si $p < 20\%$: el monto se cancela. Con la CV aleatoria, ese corte teórico es también el óptimo de la curva de ganancia (78,9%); con 15% se obtiene 78,7%, una diferencia que no es significativa (IC95 por bootstrap de $-0,3$ a $+0,7$). Un umbral por tramo de monto o por país, elegido con el pasado, tampoco mejora de forma concluyente en la última semana. El 50% que usa por defecto un clasificador, en cambio, deja 4,7 puntos en la mesa.

POLÍTICA ELEGIDA • UMBRAL 20%

78,9%

de la ganancia ideal

CORTE POR DEFECTO • 50%

74,2%

el de un clasificador: aprueba demasiados fraudes

DIFERENCIA

-4,7 pp

con el mismo modelo

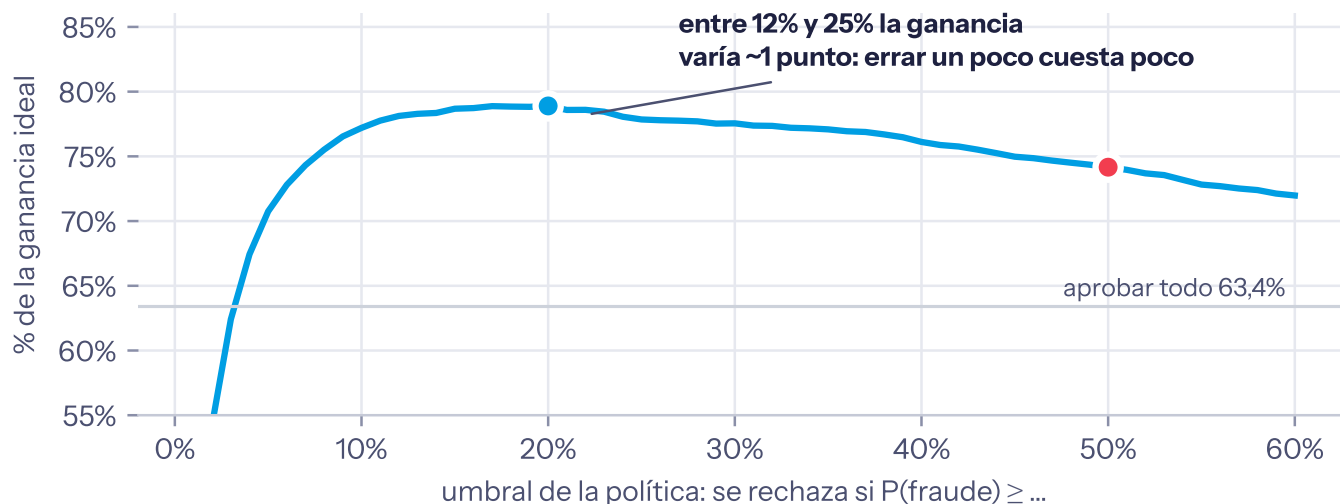


Figura 9 · Ganancia según el umbral; en celeste la política elegida, en rojo el 50%.

Las probabilidades están calibradas (en el decil más alto, 31% predicho frente a 31% observado; cerca del umbral, 17% predicho frente a 19% observado ponderando por monto), lo que permite razonar el umbral con la función de costo. **En el tiempo, la evidencia apunta a un umbral más bajo, pero no se sostiene:** con 15%, cada una de las tres últimas semanas gana entre 0,5 y 1,6 puntos, y cerca del umbral el modelo subestima el fraude. Elegido como en producción, solo con las semanas previas, el umbral sale 0,12, y en la semana siguiente no se distingue de 0,20 (+0,1 puntos, IC95 de -1,7 a +2,3; con una semana, la prueba no tiene potencia). La CV aleatoria en esas mismas semanas también elige uno bajo (0,11): son semanas con más fraude, y el óptimo semanal se mueve entre 0,12 y 0,17. Se mantiene 0,20, que sale de los costos y no hay que estimarlo. Como el óptimo baja cuando sube la tasa de fraude, lo que se ajusta en producción son las probabilidades, corrigiéndolas por la tasa vigente, no el umbral.

Del modelo al dinero



Figura 10 · Resultado sobre las 150.000 transacciones, con predicciones fuera de muestra.

Composición de la ganancia

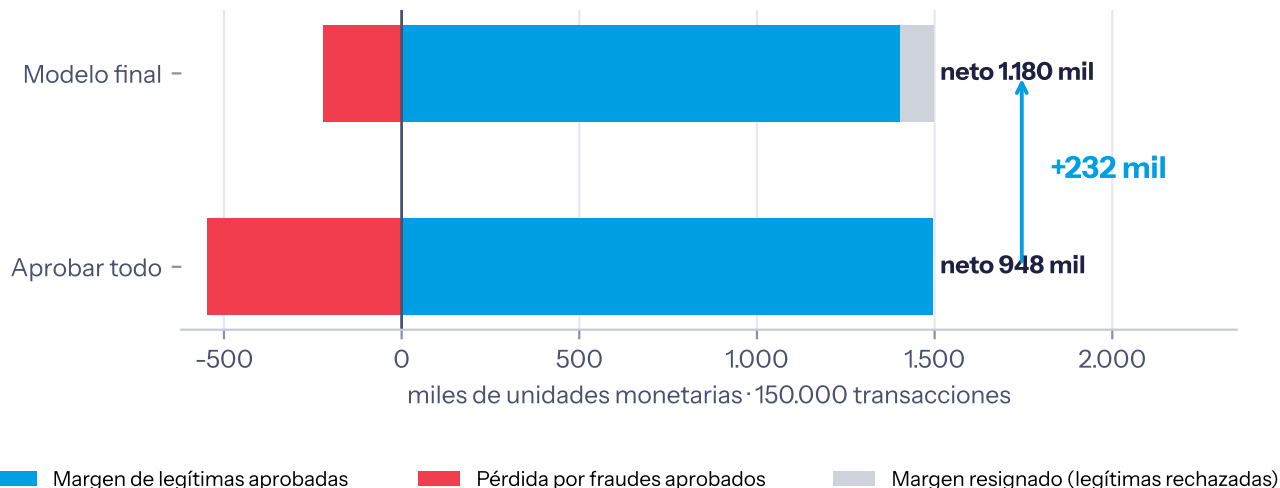


Figura 11 · Se resignan 93 mil de margen para reducir la pérdida por fraude de 547 a 222 mil.

Robustez

- **Entre folds** la ganancia varió entre 78,1% y 81,1%: diferencias menores a ~1 punto no se tomaron como concluyentes.
- **Sin el score** el baseline pierde 1,3 puntos (76,4%) y sigue muy por encima de las reglas actuales.
- **En el tiempo**, entrenando solo con el pasado, el modelo final da 75,3%, 73,7%, 77,1% y 79,5% en las cuatro últimas semanas: -1,2 puntos en promedio frente a la CV aleatoria en esas mismas filas. Casi toda la brecha es de la primera (-3,4), entrenada con solo 60.000 transacciones; en las otras tres, -0,4. Con hiperparámetros y umbral elegidos solo con el pasado, la última semana da 80,0%: elegirlos con todos los datos no infló el resultado.
- **Entre semanas** la ganancia varía más que entre folds (desvío de 2,5 puntos frente a 1,3), y la CV aleatoria varía igual en esas filas: es la dificultad de cada semana. En producción cabe esperar **~77%, entre 74% y 80% según la semana**.

¿En qué se apoya el modelo?

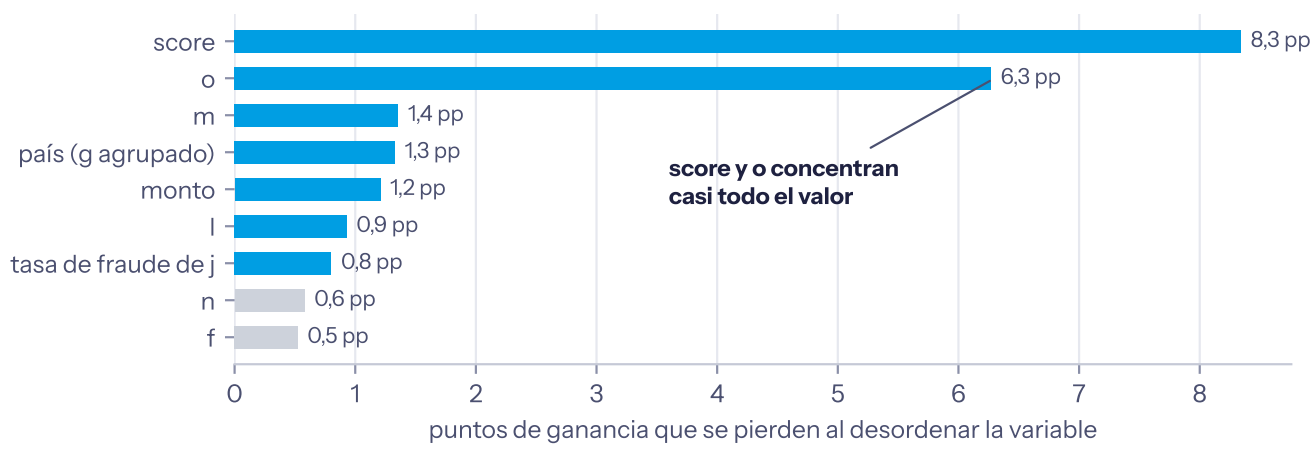


Figura 12 · Importancia por permutación medida en ganancia; en celeste, las que valen más de 0,8 puntos.

Las relaciones aprendidas son coherentes con las hipótesis (la U del score, la madrugada, los montos altos, o informado). Sin el perfil o/n/p, o concentra esa señal y es la segunda variable en valor.

Los errores más caros

CASO	MONTO	SCORE	P(FRAUDE)	DECISIÓN	EFEECTO	POR QUÉ
Fraude detectado	1.656	89	77%	Rechazar	+1.656 evitado	Score, o y monto apuntaban al fraude
Fraude no detectado	1.682	76	12%	Aprobar	-1.682	Bajo el umbral: score medio
Legítima rechazada	2.878	94	31%	Rechazar	-719 de margen	Score muy alto, monto alto, país de más riesgo

Los dos errores caros tienen la misma raíz: la dependencia del score. Los fraudes con score medio son el principal punto ciego.

El pipeline que se evaluó es el que decide

HALLAZGO

La validación, el scoring batch y la API usan el mismo código: el 78,9% no depende de un notebook.

Pregunta 3 • Cómo lograr que el laboratorio se parezca a producción

- Ajustar las variables con estado solo en entrenamiento**
Tasa y frecuencia de `j` y países frecuentes se aprenden en cada fold; así validado, el pipeline reproduce $78,9\% \pm 1,3$.
- Separar la partición de búsqueda de la de evaluación**
El tuning se volvió a medir con otra partición: $+0,4$ frente a $+0,7$. Esa partición decidió también variables y umbral; la prueba limpia es la última semana, con todo elegido solo con el pasado (80,0%).
- Medir en ganancia y con su margen de error**
 $\pm 1,3$ puntos entre folds, pero $\sim 2,5$ entre semanas: las alertas deben usar la variación semanal para no confundir ruido con degradación.
- Usar el mismo código en ambos lados**
Un único pipeline, con tests que verifican que una transacción recibe la misma probabilidad sola, en lote o por la API.
- Confirmar supuestos y probar en sombra**
Verificar que el score existe antes de decidir, comparar distribuciones y tasa de fraude reales, y correr el modelo en paralelo sin decidir antes de activarlo.

Pregunta 4 • Causas más probables si producción difiere mucho

CAUSA	QUÉ PASA Y CÓMO SE RELACIONA CON ESTE PROYECTO
1. Información que no existe al decidir	Una variable, como el score, se calcula con datos posteriores. Es la causa típica de caídas bruscas y el supuesto crítico de este modelo.
2. Variables calculadas distinto	Tipos, nulos o categorías nuevas se tratan distinto al servir. Ocurrió dos veces durante la construcción; por eso existen los tests de consistencia.
3. Cambia la tasa de fraude	Las probabilidades dependen de la tasa base: hay que corregirlas por la vigente. El 5% exacto sugiere muestreo y la tasa diaria varió entre 2,2% y 6,6%.
4. Cambian los patrones	Los defraudadores se adaptan y los datos son de un período atípico (inicio de la pandemia).
5. Cambian las entradas	Categorías de <code>j</code> nunca vistas o una recalibración del score en el sistema que lo produce.
6. Se mide distinto	Contracargos que llegan semanas después, rechazadas sin etiqueta y semanas cuya ganancia varía $\sim 2,5$ puntos entre sí.

Pregunta 5 • Pasos para ponerlo en producción

- Empaquetar como pipeline reproducible** HECHO
Variables, modelo y política en un único artefacto; 89 tests con 100% de cobertura, en CI.

2 Validar, registrar y versionar HECHO

Cada validación y entrenamiento queda en MLflow; la versión en producción lleva el alias `champion`, que solo se mueve si pasa un gate de promoción, y el rollback es mover ese alias.

3 Servir HECHO

API en Cloud Run con validación del input y API key; la imagen incluye el modelo. También scoring batch.

4 Desplegar gradualmente SIGUIENTE

Modo sombra hasta tener etiquetas, luego un porcentaje creciente del tráfico con criterios de éxito definidos. Latencia medida: 28 ms p99 en el servidor.

5 Monitorear y reentrenar SIGUIENTE

Tasa de rechazo, distribución de probabilidades y drift de variables sin esperar etiquetas; ganancia real cuando lleguen. Reentrenar y revisar el umbral cuando cambien.

Arquitectura: separar modelo, pipeline, decisión y operación

OFFLINE · ENTRENAMIENTO

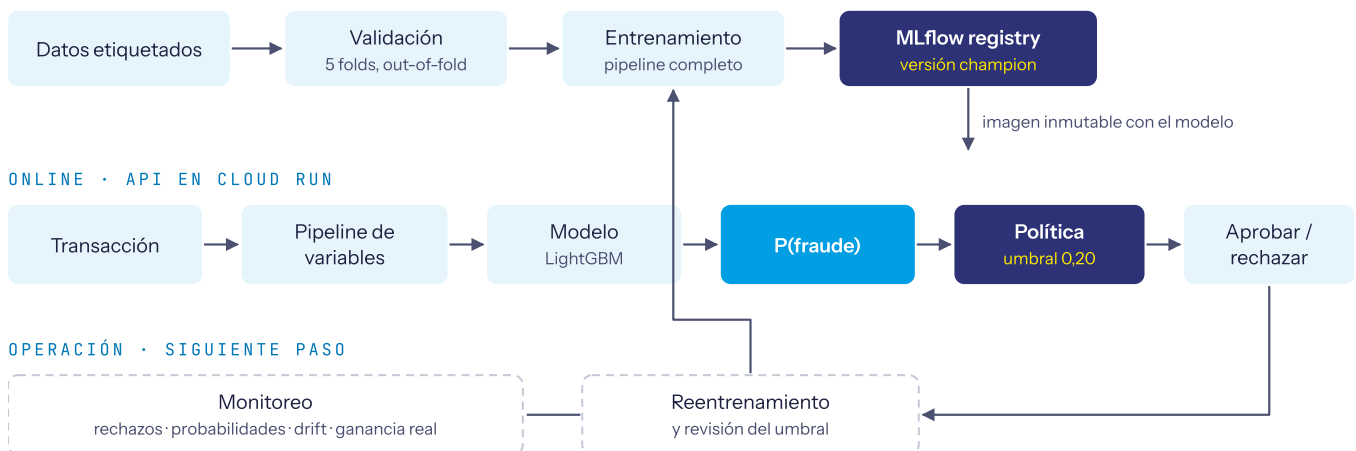


Figura 13 · Implementado: capas offline y online (MLflow en Cloud Run con Supabase y Cloud Storage). Pendiente: operación.

Detectar el fraude **que más cuesta**

El sistema alcanza el 78,9% de la ganancia ideal: +15,5 puntos sobre aprobar todo y +11,5 sobre la regla del score, equivalentes a 232 mil más sobre las transacciones analizadas, evitando el 59% del dinero perdido por fraude y aprobando el 94,5% de los pagos. El valor no vino de un algoritmo en particular, sino de medir en dinero, representar bien la historia de las categorías, decidir con el costo de cada error y medir el optimismo de la selección, también en el tiempo.

Lo que haría en producción

1 Derivar el corte del costo

Frente al 50% por defecto vale 4,7 puntos con el mismo modelo. Dejarlo fijo y, si cambia la tasa de fraude, corregir las probabilidades en lugar de mover el umbral.

2 Mantener **j** como historial, no como categoría

La tasa suavizada y out-of-fold conserva la señal; la variable cruda empeora el modelo.

3 Confirmar y vigilar el score

Verificar que existe antes de decidir y monitorear su distribución: es la pieza de la que más depende el resultado.

4 Empezar en modo sombra

Comparar la ganancia real con la del sistema vigente antes de dejar que el modelo decida.

5 Revisar primero los fraudes con score medio

Son el principal punto ciego y el lugar más prometedor para la próxima mejora.

Limitaciones

- **Variables anonimizadas:** las lecturas de negocio (historial, perfiles, país) son interpretaciones. El monto parece estar en una moneda común: su escala es similar entre países (mediana 18 en Brasil, 29 en Argentina).
- **Datos muestreados y atípicos:** 5% exacto de fraude y 45 días del inicio de la pandemia; con la tasa real, las probabilidades deben corregirse por la tasa base.
- **Sesgo de selección y fricción:** si el dataset excluye lo que el sistema actual ya rechaza, el modelo no vio ese tráfico. Y la matriz de costos no mide la fricción de rechazar 4.612 compras legítimas (abandono, reclamos): en la franja cercana al umbral conviene revisar o pedir autenticación (3DS) en lugar de rechazar.
- **Validación temporal corta:** cuatro semanas de un período atípico. Quitar el perfil o/n/p se decidió con todos los datos, incluida la última semana: el holdout temporal es limpio para hiperparámetros y umbral, no para esa decisión.

Detectar fraude optimizando la ganancia, y medir con honestidad cuánto del resultado depende del feature engineering, de la validación y del despliegue.

LA IDEA CENTRAL DEL PROYECTO

Glosario, ficha del modelo y reproducibilidad

Glosario

Ganancia ideal (máxima posible)

25% de la suma de los montos legítimos: aprobar todas las ventas legítimas y ningún fraude.

Modelo y política

El modelo estima $P(\text{fraude})$; la política decide aprobar o rechazar con un umbral.

Predicción out-of-fold

Predicción de una transacción hecha por un modelo que no la usó para aprender.

Target encoding

Reemplazar una categoría por su tasa histórica de fraude, suavizada hacia la media.

Calibración

Que una probabilidad de 30% signifique 30 fraudes de cada 100 casos similares.

Importancia por permutación

Ganancia perdida al desordenar una variable.

ROC-AUC / PR-AUC

Capacidad de ordenar por riesgo (general y con pocos positivos). No miden la ganancia.

Ficha del sistema

MÉTRICA DE NEGOCIO

78,9% ± 1,3

de la ganancia ideal · CV de 5 folds (semilla 7) · por semana, 74–80%

MÉTRICAS DE ML

ROC-AUC 0,889 · PR-AUC 0,474

Modelo	LightGBM · <code>n_estimators</code> 481 · <code>learning_rate</code> 0,019 · <code>num_leaves</code> 172 · <code>min_child_samples</code> 262 · <code>subsample</code> 0,96 · <code>colsample_bytree</code> 0,96 · <code>reg_alpha</code> 1,61 · <code>reg_lambda</code> 3,03
Variables	<code>a</code> <code>b</code> <code>c</code> <code>d</code> <code>e</code> <code>f</code> <code>h</code> <code>k</code> <code>l</code> <code>m</code> <code>n</code> <code>p</code> <code>monto</code> <code>score</code> + <code>g_agrupado</code> , <code>hora</code> , <code>j_frecuencia</code> , <code>j_tasa_fraude</code>
Target encoding	Suavizado de 20 hacia la tasa global; out-of-fold en entrenamiento
Política	Rechazar si $P(\text{fraude}) \geq 0,20$
Referencias	Aprobar todo 63,4% · <code>score</code> ≥ 90 : 67,4% · baseline 77,7%

Reproducir

Código en github.com/JuanFeDS/proyecto (Python 3.12, Poetry):

```
poetry install ·
scripts/validate_pipeline.py ·
scripts/train.py · scripts/predict.py
```

Los notebooks 01–06 tienen el análisis completo; MLflow con los ~70 experimentos y la API están desplegados en Google Cloud Run (acceso a solicitud).